🟤 CodeChef Scoring — Deep Dive (40 pts total)

What data we fetch from CodeChef

When the scraper visits `codechef.com/users/<username>`, it extracts:

Field	Source in HTML	What it means
`current_rating`	`all_rating` JS array → last entry's `.rating`	Your CC rating after your most recent contest
`contest_count`	`all_rating` array length	Total contests you've participated in
`problems_solved`	`<h3>Total Problems Solved: N</h3>`	Problems solved in practice section
`stars`	Computed from rating	Your division tier (1★–7★)

Part 1 — CC Problem Score (20 pts)

This measures **how much you've practiced** on CodeChef.

Step 1: Raw Points

```
raw_pts = problems_solved × 4
```

Why `× 4`? CodeChef problems are mostly **Division 3–4 level** (medium difficulty). In the UDG tier system, a Tier-3 problem = 4 points. So we assume every CC problem solved ≈ 4 UDG pts on average.

Example:

```
Student solved 193 problems (like devi_996)
raw_pts = 193 × 4 = 772
```

Step 2: Consistency Factor

Since we don't have week-by-week submission history, we apply a penalty based on volume — more problems = more consistent:

Problems solved	Factor
≥ 400	0.80
≥ 200	0.70
≥ 100	0.60

Problems solved	Factor
< 100	0.50

Example: 193 problems → factor = **0.70**

```
effective = 772 × 0.70 = 540.4
```

Step 3: Ratio against Benchmark

The benchmark is **350 UDG pts** (what a "100%" CC student should have):

```
ratio = clamp(540.4 / 350, 0, 1) = clamp(1.544, 0, 1) = 1.0  (capped at 1)
```

Step 4: Power Curve

```
prob_score = 20 × ratio^0.7
```

The `^0.7` exponent makes the curve **concave** — meaning students in the middle range aren't unfairly penalized too hard. Without it (linear), getting from 50% to 100% is as hard as getting from 0% to 50%. With `^0.7`, mid-range students get more credit.

```
prob_score = 20 × 1.0^0.7 = 20 × 1.0 = 20 pts  (maxed out)
```

Another example — 50 problems solved:

```
raw_pts   = 50 × 4 = 200
effective = 200 × 0.50 = 100   (low volume → factor = 0.50)
ratio     = 100 / 350 = 0.286
prob_score = 20 × 0.286^0.7 = 20 × 0.396 = 7.9 pts
```

Part 2 — CC Contest Score (20 pts)

This measures **how actively you compete** and **how highly rated you are**.

Split into two sub-components:

```
contest_score = 20 × (0.40 × P + 0.60 × L)
                       ↑ Participation   ↑ Level
```

P — Participation (40% weight of contest part)

```
P = min(contest_count / 18, 1.0)
```

Expected benchmark = **18 contests** (roughly 1 per month over ~1.5 years). Beyond 18, P is capped at 1.0.

Contests	P value
0	0.00
5	0.28
10	0.56
18	1.00
30	1.00 (capped)

L — Level Score (60% weight of contest part)

Based on **star rating**, not raw rating number:

CC Rating	Stars	L value
0 (never rated)	0★	0.00
1000–1199	1★	0.10
1200–1399	2★	0.25
1400–1599	3★	0.45
1600–1999	4★	0.65
2000–2499	5★	0.80
≥ 2500	6★+	0.95

Why stars instead of raw rating?
Rating numbers on CC aren't continuous — a 1★ (1050 rating) and a 2★ (1200 rating) are vastly different in skill, but the number difference is only 150. Stars are a better tier representation.

Full Worked Example

Student: `mallidimounika`

- `problems_solved = 586`
- `contest_count = 24`
- `current_rating = 1228` → **3★** → L = 0.45

Problem Score:

```
raw_pts   = 586 × 4 = 2344
factor    = 0.80  (≥ 400 problems)
effective = 2344 × 0.80 = 1875.2
ratio     = min(1875.2 / 350, 1) = 1.0  (capped)
prob_score = 20 × 1.0^0.7 = 20.0 pts
```

Contest Score:

```
P = min(24 / 18, 1) = 1.0
L = 0.45  (3★)
contest_score = 20 × (0.40 × 1.0 + 0.60 × 0.45)
              = 20 × (0.40 + 0.27)
              = 20 × 0.67
              = 13.4 pts
```

Total CC Score:

```
CC = 20.0 + 13.4 = 33.4 / 40
```

Key Insight: What hurts your CC score the most?

Issue	Impact
Never participated in rated contests	L = 0, contest part heavily penalized
Low stars (1★ or 2★)	L = 0.10 or 0.25 — contest score capped low
Few problems solved (< 50)	prob_score barely above 7–8 pts
Non-existent/private CC profile	CC score = **0** (entire 40 pts lost)

The **single biggest lever** is your **star rating** — going from 2★ → 3★ (1200→1400 rating) moves L from 0.25 → 0.45, which alone adds ~`20 × 0.60 × 0.20 = 2.4 pts` to your score.